Statement of Academic Integrity Regarding Plagiarism

I, the undersigned Tazi Rachid [family name, given name(s)], hereby certify on my honor that:

1. The results presented in this report are the product of my own work.

2. I am the original creator of this report.

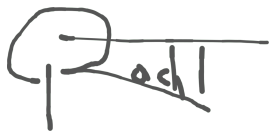
3. I have not used sources or results from third parties without clearly stating thus and referencing them according to the recommended rules for providing bibliographic information.

Declaration to be copied below:

I hereby declare that this work contains no plagiarized material.

Date 16/03/2026

Signature





École Polytechnique

BACHELOR THESIS IN COMPUTER SCIENCE

Exploring text-based generative AI for creating manipulable physically-based scenes in Unity 3D.

Author:

Tazi Rachid, École Polytechnique

Advisor:

Marie-Paule Cani, École Polytechnique

Academic year 2025/2026

Abstract

This project explores to what extent text-based generative AI such as ChatGPT can be used in the process of generating Unity C# scripts and automate the configuration of physically manipulable objects. We designed a prompting pipeline to generate Unity scripts from a textual discussion. Objects were organized into families and configured using predefined presets. An AI agent was developed to iteratively propose improvements to physical parameters. These results highlight both the potential and current limitations of LLMs for automated scene generation in Unity.

Contents

1 Introduction **4**
1.1 Context . . . 4
1.2 Problem . . . 4
1.3 Objectives . . . 5

2 Learning and Exploration of Unity **5**
2.1 Introduction to Unity . . . 5
2.2 GameObjects and Components . . . 6
2.3 Unity Physics engine . . . 6

3 State of the Art **8**
3.1 Physics-Based interaction in virtual environments . . . 8
3.2 Rigging and Skeleton Generation for 3D meshes . . . 8
3.3 AI as a tool for Unity code generation . . . 9
3.4 LLM-based Agents for 3D scene generation . . . 10

4 Proposed approach **11**
4.1 Manual exploration of LLMs . . . 11
4.2 Structured prompt template for guided script generation . . . 12
4.3 Object family scripts and physical presets . . . 13
4.4 AI agent for iterative physical authoring . . . 16

5 Results **19**
5.1 Manual Unity setup . . . 19
5.2 Direct LLM generation . . . 19
5.3 Structured prompt template . . . 19
5.4 Object family scripts and physical presets . . . 19
5.5 Iterative AI agent . . . 20

6 Future works **20**

7 Conclusion **20**

8 References **22**

A Appendix **23**
A.1 Example of generated Unity script . . . 23
A.2 Prompt template for guided physical authoring . . . 24
A.3 AI Agent Implementation . . . 32
A.3.1 Main execution file (`main.py`) . . . 32
A.3.2 Iteration loop (`loop.py`) . . . 34
A.3.3 Mesh loading and preprocessing (`mesh_read.py`) . . . 36
A.3.4 Prompt construction (`prompting.py`) . . . 38
A.3.5 Model client (`model_client.py`) . . . 41
A.3.6 Rendering module (`render_pyrender.py`) . . . 46
A.3.7 Reporting utilities (`report.py`) . . . 48
A.3.8 Contract schema and validation (`schema.py`) . . . 50

1 Introduction

1.1 Context

In the current context, generative AI is booming. Since the 2020s, models have become increasingly powerful and have been trained on massive datasets. We have seen an incredible improvement in the models' reasoning capabilities across a wide range of situations.

These advances in AI have led to a significant increase in the number of trained models over the past few years. Chat GPT, Gemini, Claude and many other models are becoming essential tools in our daily lives. Whether it's business, developers, students, or researchers, everyone uses it.

Large language models (LLMs) are among the most influential generative AI systems developed in recent years. These models are deep neural networks trained on massive text datasets using self-supervised learning techniques. During training, these models learn statistical patterns and relationships within language, enabling them to answer questions, summarize documents, translate languages, or even generate code.

The ability to interpret natural language instructions and transform them into structured outputs allows them to integrate information from different types of data such as text, images, or videos to perform a wide range of tasks.

One of the most promising applications of generative AI is automatic code generation. The arrival of generative AI has had a huge impact on the field of computer science, providing assistance in code generation in most available languages. Indeed, LLMs can produce scripts, functions, or entire program structures from natural language descriptions. AI-based conversational assistants such as Github Copilot help boost productivity and workflows by offering contextual assistance and automating routine tasks. However, code generation with LLMs still presents some limitations. The code generated may contain inconsistencies and rely on incorrect assumptions. When we apply it in environments that require precise structural or physical properties, these limitations become particularly challenging.

3D interactive environment scenes are often created using game engines such as Unity or Unreal Engine. A game engine is a set of software components that perform geometry and physics calculations. It can be used to create a real-time simulator that simulates the characteristics of the virtual worlds. These 3D engines can be used in a variety of fields, such as video games, virtual reality, scientific simulations, and data visualization. Unity is a particularly flexible platform that allows users to create interactive scenes by combining graphic elements with physical behaviours.

To give 3D objects realistic physical behaviour, several carefully chosen parameters must be configured. These parameters influence how objects behave in the environment. Nowadays, these configurations are done manually by developers based on the geometry and real-world characteristics of the object. Thus, creating physically plausible objects can require both experience and manual effort. Manual configuration presents some limitations. First, it is a time-consuming process, especially when environments contain many objects. Second, the physical properties chosen by the developers may lack consistency, since they are often chosen arbitrarily, which can lead to inconsistent behaviour.

1.2 Problem

Given recent advances in generative AI and code generation, it is worth considering whether **generative AI can assist us in automatically creating physically interactive objects in a 3D engine such as Unity.** AI models

have proven their effectiveness in code generation and reasoning, but it remains to be seen how well they can be used to produce stable physical behaviours for arbitrary 3D assets.

The objective of this project is to explore text-based generative AI for creating manipulable, physically-based scenes in Unity 3D. The goal is to investigate how AI can help automate the configuration of physical parameters and object behaviours. In particular, the work seeks to evaluate to what extent AI models are capable of analyzing a 3D object, infer its structural properties, and propose coherent and realistic physical parameters. Another question addressed in this project is whether these models can automatically generate Unity scripts that configure the physical behaviour of objects within the engine.

1.3 Objectives

As a beginner with a strong interest in computer graphics, the primary goal is to learn about 3D animation and understand how objects are represented and animated in Unity. In addition, the goal is to explore the capabilities of large language models to generate scripts usable by Unity. Another objective is to design an automated pipeline capable of assisting the configuration of physical behaviours for 3D objects, and at the end to create a small interactive virtual world composed of physically manipulable objects.

To address this problem, several approaches are explored throughout this work. This includes the design of pipelines for generating physical configurations of objects, as well as a series of experiments aimed at evaluating the effectiveness, stability and practical usability of these approaches.

2 Learning and Exploration of Unity

2.1 Introduction to Unity

Unity is a widely used game engine that allows the creation of interactive 3D environments. It offers a visual editor where we can freely create scenes by placing objects and configuring behaviours. In this project, Unity is the primary platform on which all our approaches are implemented. The goal of this exploration phase is to discover the game engine and understand how objects are represented in Unity.

The first step of this exploration consisted of following a tutorial to create a simple platformer game. I learned how to create platforms and spheres, and how to control an object's movement using inputs.

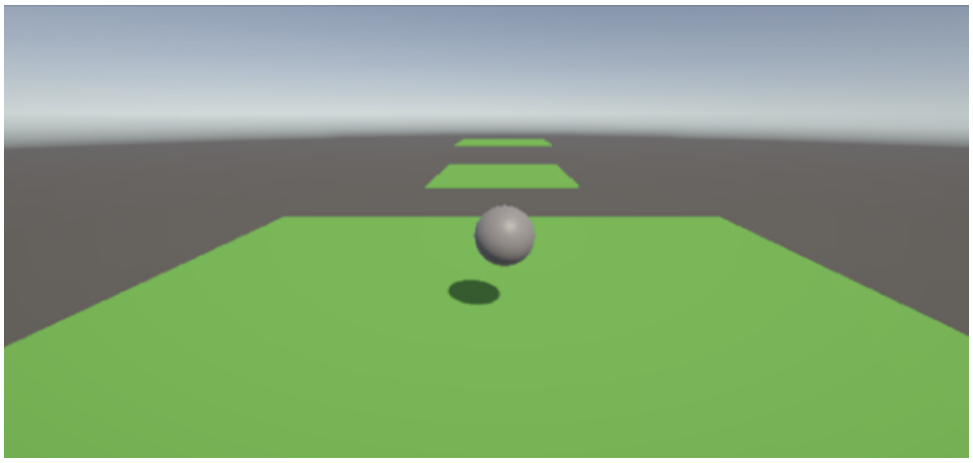


Figure 1: Example of a first scene created in Unity.

Here is the link to the tutorial: https://www.youtube.com/watch?v=n0GQL5JgJcY&list=PLrnPJCHvNZuBAKuHOxFnV55wnkqKK8gqL

2.2 GameObjects and Components

Everything in the scene is called a GameObject. These are fundamental entities and can represent all kinds of elements, such as 3D models, lights, cameras, or interactive objects. These GameObjects have a component-based architecture. Indeed, their behaviour and properties are defined by attaching components to them. For example, a visual object requires a MeshFilter and a MeshRenderer component to display a 3D model.
Multiple components can be attached to a single GameObject.

2.3 Unity Physics engine

Unity includes a physics engine that simulates physical interactions between objects. This system allows objects to move, collide, and respond to forces. The Rigidbody component enables objects to participate in the physics simulation. It defines properties such as mass, damping, or gravity, which can influence the motion of the object.

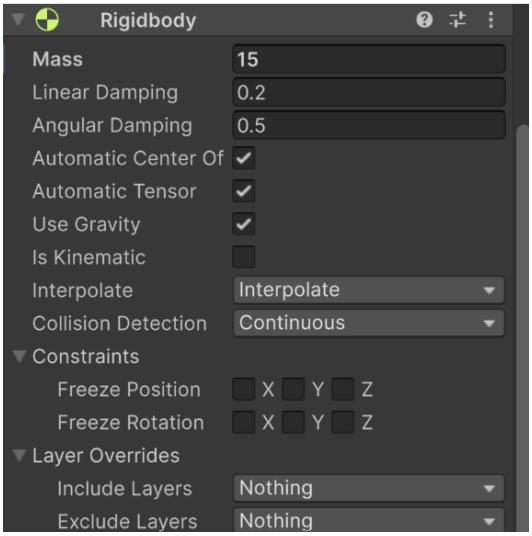


Figure 2: Inspector view of the Rigidbody component in Unity.

Colliders define the physical shape of an object used for collision detection. A 3D object needs a collider to interact with the game world. Unity offers various types of colliders: box colliders, sphere colliders, capsule colliders, and mesh colliders. Each type is best suited for certain objects; for example, for a ball, you would choose a sphere collider. Unlike primitive colliders, the mesh collider uses the exact geometry of the 3D model to define the collider's shape, this can be computationally more expensive which can provide more accurate collision representations for more complex objects. However, this approach is computationally more expensive. Here is an example of a Rock with a mesh collider:

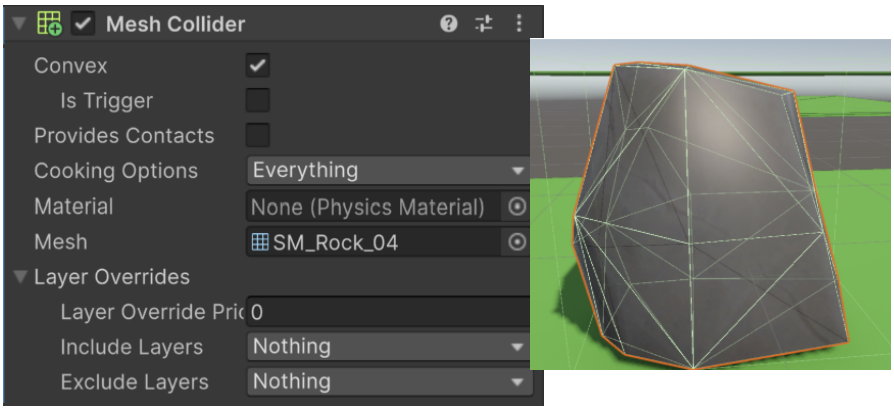


Figure 3: Example of a mesh collider applied to a rock object in Unity.

Physics Materials allow developers to control surface properties such as friction and bounciness. They influence how objects slide, bounce or roll during collisions.

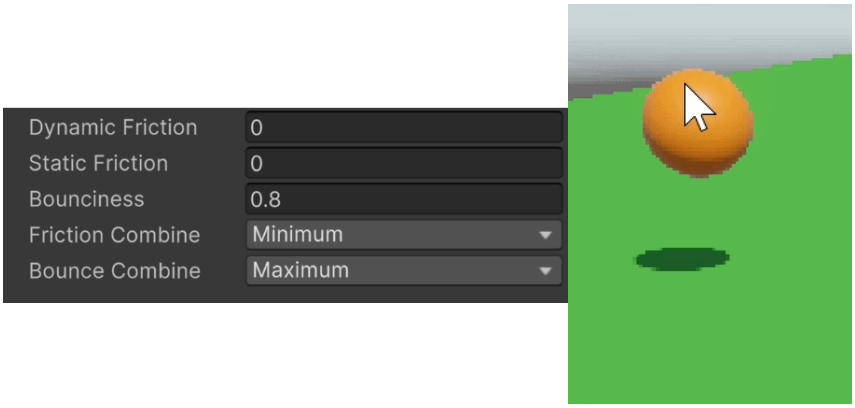


Figure 4: Example of a Physic Material in Unity, used to define friction and bounciness properties.

In addition, we can group different GameObjects together using joints. This allows us to create articulated objects with multiple parts.

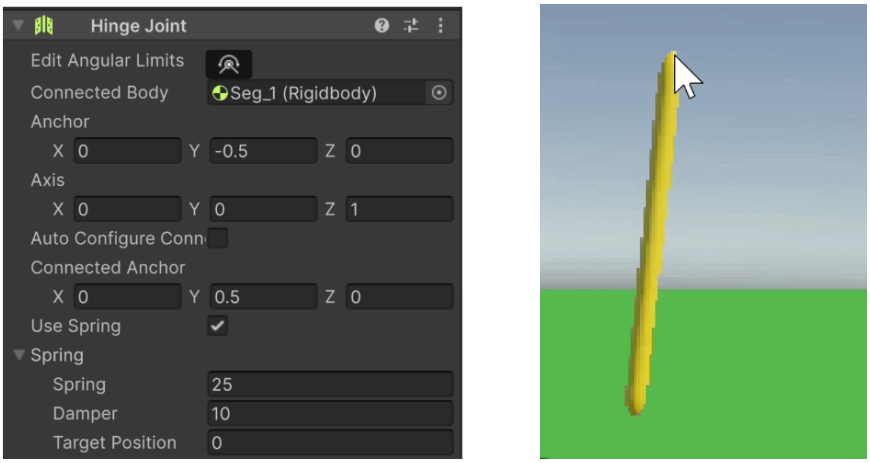


Figure 5: Example of a hinge joint in Unity used to connect multiple parts and create an articulated object.

Understanding all these elements is necessary in order to set up realistic, interactive objects in an environ-

ment. This will allow us to apply our experiences and various approaches to investigate to what extent LLMs can contribute to the generation of components for physical manipulable objects.

3 State of the Art

In order to fully explore our research question, it is necessary to review existing research on the topic.

3.1 Physics-Based interaction in virtual environments

To better understand the challenges addressed in this work, it is important to recognize that certain physical interactions can be challenging and time-consuming to implement in virtual environments. Reproducing stable interactions can pose a major challenge. To deliver believable experiences, virtual objects must behave according to consistent physical laws when they are manipulated or when they interact with other elements of the environment. To solve this problem, researchers have increasingly turned to physics-based interaction systems capable of simulating forces, collisions, and friction between objects. One example is the work by Kiran Nasim and Young J. Kim [2], which addresses the difficulties in achieving stable and realistic object manipulation in virtual reality by proposing a physics-based grasping assistance system for object manipulation in VR. Their method introduces a "proxy hand" system that acts as an intermediary between the user's tracked hand and the objects. It can handle collision detection and physical constraints.



Figure 6: Comparison between a virtual hand grasp and a proxy hand grasp. The proxy hand prevents object penetration and allows more stable interactions between the hand and virtual objects (Nasim and Kim, 2018)

Creating physically interactive objects is a complex task that generally requires careful manual design of physical parameters and interaction systems. This demonstrates the potential value of approaches such as the one explored in this work, which aim to simplify and automate the configuration of interactive objects in a virtual world.

3.2 Rigging and Skeleton Generation for 3D meshes

Another major challenge in creating interactive 3D objects is creating articulated structures from 3D meshes. This process is called rigging. It involves creating an internal skeleton within a 3D mesh, which allows objects to be animated and deformed as the skeleton moves. However, performing this essential step manually can be time-consuming and complex, especially for complex meshes with numerous branches. Several research studies have proposed methods for automating skeleton generation from an object's geometry. A well-known example is the work of Baran and Popović(2007) [1], which proposes a geometric method for automatically animating characters. Starting from a static character mesh and a generic skeleton, their method adapts the

skeleton to the character and attaches it to the surface. They called their prototype Pinocchio and here is a picture that illustrates the automatic process proposed by the authors.

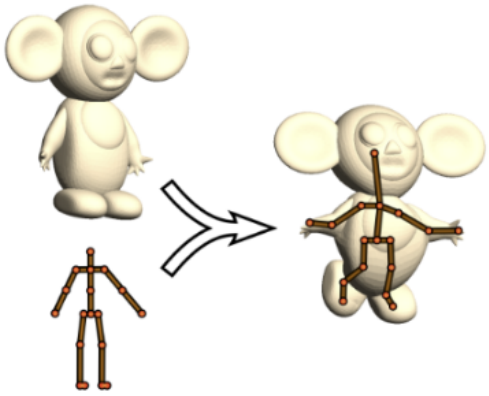


Figure 7: Automatic skeleton generation using the Pinocchio method. A generic skeleton is adapted and attached to the surface of a static character mesh to enable animation (Baran & Popović, 2007).

More recently, some research has utilized machine learning methods. For example, RigNet [4] uses neural networks trained on datasets of articulated characters to automatically predict appropriate rigs and assign blend weights.

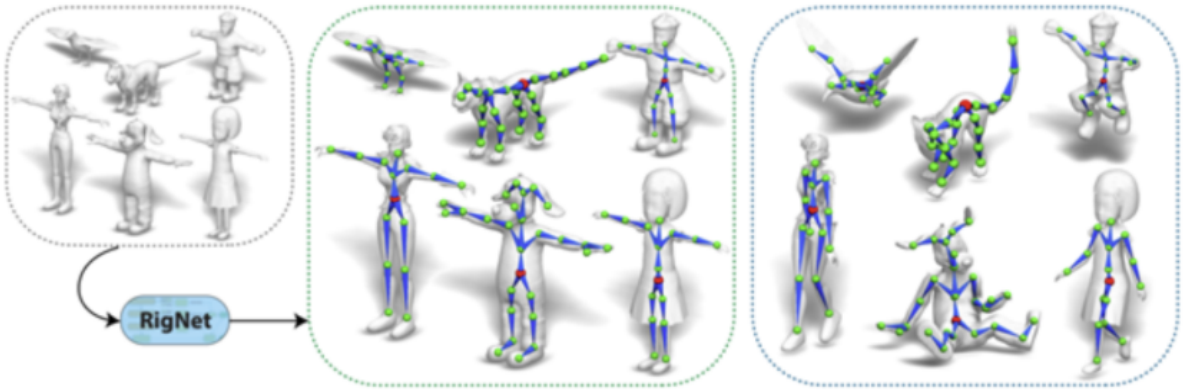


Figure 8: Automatic rigging results produced by RigNet, which predicts skeletons and skinning weights for articulated 3D characters (Xu et al., 2020).

However, these methods are often designed for specific types of objects, such as human characters or animals. It is still difficult to apply these methods to more general shapes or natural structures with branches, such as a tree. This suggests that it may be worthwhile to explore approaches capable of assisting or automating the rigging process for 3D objects to simplify the creation of interactive virtual environments.

3.3 AI as a tool for Unity code generation

To successfully carry out our approaches, it is important to explore how we could use LLMs to generate code for Unity and understand how well it performs. The work of Vivas Zamora (2024-2025) [3] is an example that

evaluates how modern AI tools can support the technical aspects of solo game development. To do this, he designed and implemented a 3D dungeon-crawling game in Unity, where AI coding was used to develop the game's systems such as movements, combats, destructible objects and AI enemies.

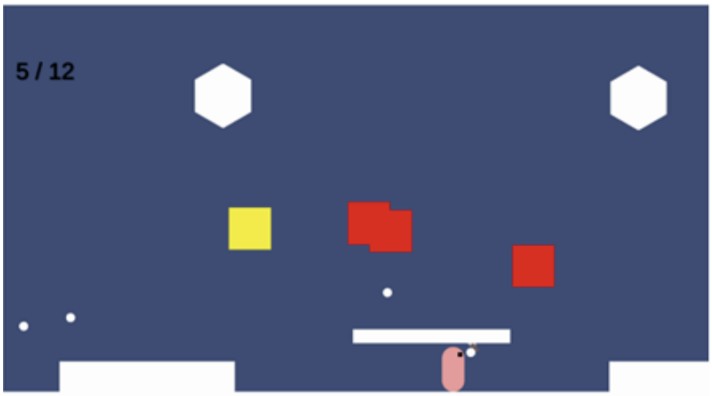


Figure 9: Simple Unity game used to compare the code generation capabilities of several AI models.

This prototype game was used to evaluate several AI models (ChatGPT, Copilot, Gemini and DeepSeek) in similar circumstances. He demonstrated that these models demonstrate great technical mastery and also concluded by saying that ChatGPT and Copilot showed better consistency, speed, and flexibility. This work shows that generative AI can significantly help developers in writing Unity scripts. The comparisons between the different models provide useful insights to identify suitable AI tools for assisting development in our project.

3.4 LLM-based Agents for 3D scene generation

Other researchers have already explored the use of LLM in generating or modifying 3D environments by code. The work "Agent Vision-as-Inverse-Graphics via Interleaved Multimodal Reasoning" [5] by Shaofeng Yin, Jiaxin Ge, Zora Zhiruo Wang, Xiuyu Li, Michael J. Black, Trevor Darrell, Angjoo Kanazawa and Haiwen Feng, introduces a multimodal agent that reconstructs images into editable scene programs through an analysis-by-synthesis loop, using interlaced multimodal reasoning and evolving contextual memory for "vibe code" the scene, its physics and its interactions.

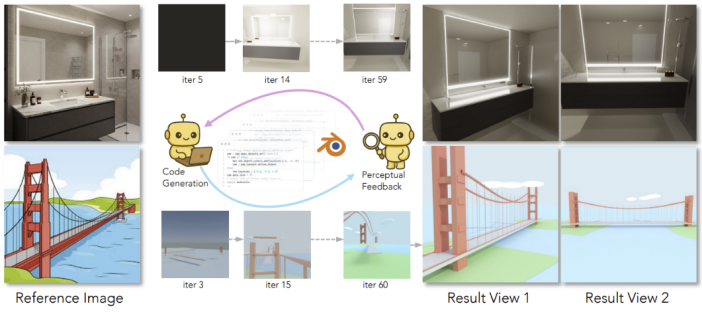


Figure 10: Multimodal agent pipeline for reconstructing a visual scene from a reference image through iterative code generation and perceptual feedback (Yin et al., Agent Vision-as-Inverse-Graphics).

It demonstrates how LLM-based agents can reason about visual environments. By combining program generation, visual feedback and memory of previous steps, such systems allow for more complex scene construction. However, these approaches mainly focus on the synthesis and editing of scenes, and do not directly

address the problem of physically coherent interaction configurations between objects in Unity. It may therefore be interesting to try to design a similar agent for our purpose.

4 Proposed approach

The objective of the project is to explore how generative AI can assist us in generating physically-based scenes in Unity. To explore this question, several approaches are gradually developed and evaluated throughout the project. Each approach attempts to address specific limitations observed in previous methods. The goal remains the same for all methods. From a 3D mesh representing an object in Unity, the objective is to infer plausible physical characteristics from the object's structure, identify a physical behavior, and generate a coherent physical configuration for the Unity-compatible object.

In the context of this project, due to time constraints, I did not compare several models. Following the results found by the paper 'AI-Assisted Game Development: Designing and Prototyping Core Gameplay Systems (2024-2025)', I decided to work mainly with ChatGPT-5.3.

4.1 Manual exploration of LLMs

The first approach is relatively simple and serves as a baseline for the following. Before trying more complex approaches, it is important to understand the limits of LLMs if we simply try to ask them to generate a C# script by giving them as input the object's mesh and screenshots. The goal is to observe if the model can, with minimal input information, generate a more or less coherent result. Here is an example of a prompt sent to ChatGPT to obtain a C# script for a 3D mesh of a stump.

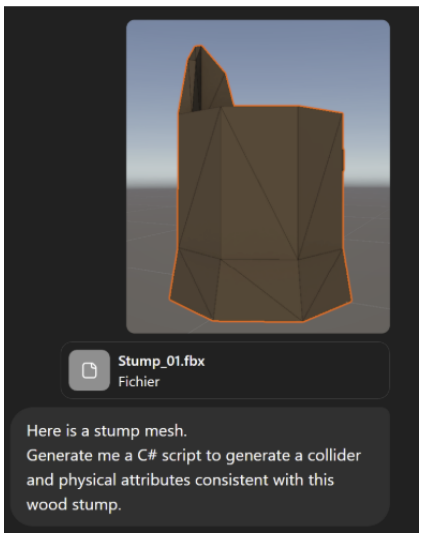


Figure 11: Example of a prompt sent to a large language model to generate a Unity C# script for a stump mesh.

The LLM produces a C# script (see ANNEX 1) which notably defines a collider and a Rigidbody:

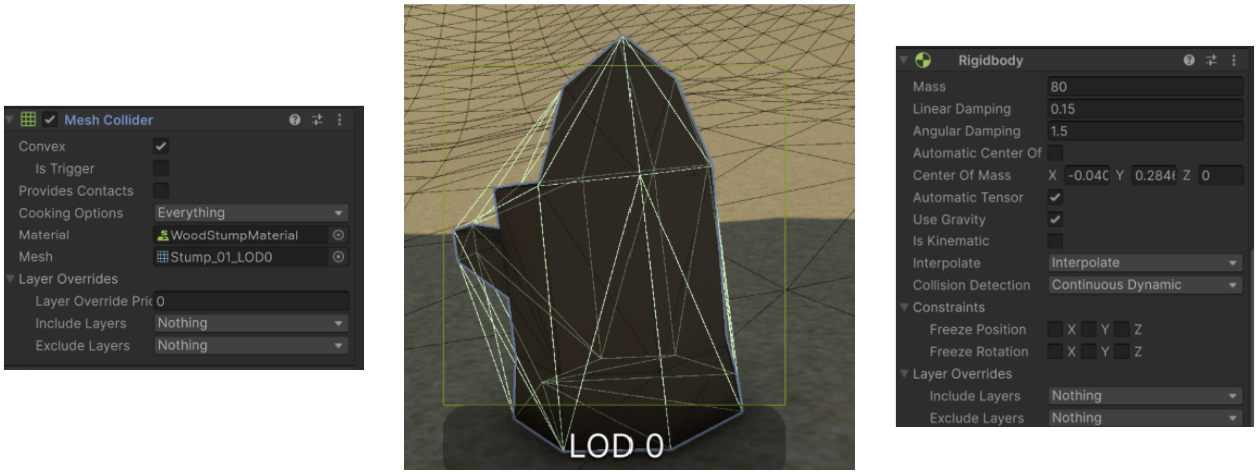


Figure 12: Configuration generated from the LLM-produced Unity script.

We then observe that LLMs can already produce C# scripts that generate a Mesh Collider, a Rigidbody, a physics material and choose physical parameters. This simple pipeline: prompt + screenshot + mesh allows to obtain a decent result. The model understands the nature of the object and has given characteristics related to a rigid object The MeshCollider allows for a precise collider around the object, but it can be computationally expensive if there is a large number of stumps in the simulation, which is why we will try to obtain simpler colliders in the next methods. We still notice that the chosen parameters are approximate because the LLM has no information about the scale of the scene and also because some values can be set or chosen heuristically:

```
rb.mass = Mathf.Clamp(estimatedMass, minMass, maxMass);
```

Figure 13: Excerpt of the generated Unity C# script showing how the object mass is estimated and constrained using heuristic bounds

The parameters are therefore relatively arbitrary and if we proceed with this method for different objects, they will not be standardized in the same way.

Moreover, it happens that the code proposed by the LLM may contain compilation errors:

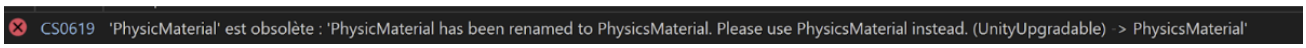


Figure 14: Example of a compilation error produced by the generated script.

This can happen when the model is using an older version of Unity, which may lead to the use of functions or APIs whose names have changed.

Finally, I also observed that the results strongly depend on the formulation of the prompt, because the LLM bases all its assumptions on the information provided in the prompt. It would therefore be necessary to design more structured prompts in order to guide the generation of the script more precisely.

4.2 Structured prompt template for guided script generation

As the direct generation of C# scripts with an LLM gives interesting results, I found it interesting to deepen this idea and try to better structure the prompts to try to obtain more stable and standardized results.

In this sense, the objective is to design and design a structured prompt template to guide the model's reasoning. The idea is to decompose the problem in several steps instead of directly asking for a complete script. Each step corresponds to a part of the process of physical configuration of an object in Unity. The steps include an object identification, a mesh structure analysis, a physical classification, a behavior choice, a collider design, an interaction definition, a choice of physical parameters and a Unity script generation.
To design the template, I followed an iterative and experimental process. After creating a first simple template to generate C# scripts, I tested it on some objects and analyzed the problems observable in Unity. Thus, in an iterative manner, I was able to improve my prompts template. Here are some results that show the evolution of the results of this method for a tree mesh:

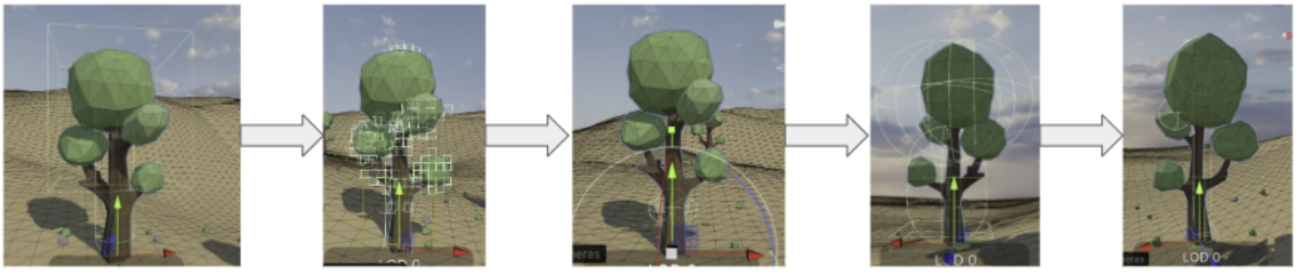


Figure 15: Evolution of the generated physical configuration for a tree mesh using progressively improved structured prompts.

We can see that the first results show several problems: the colliders are too simple, there are too many generated colliders or the colliders are poorly positioned. The template is gradually modified to limit the number of colliders, choose more suitable shapes and improve their placement. The results show a gradual improvement in the physical configuration of objects. The most advanced template I made is in the appendix.

However, several limitations appear. First, the generation pipeline is quite long for each object because it is based on a discussion between the LLM and the user. A pipeline can last between 10 and 15 minutes. Then, I noticed that the sensitivity of LLMs was also perceptible with this method; a small change in a prompt can modify the result of the entire pipeline. Finally, I happened to get different results with the same template. This shows that even with a structured template, it is difficult to fully control the randomness of LLMs.

4.3 Object family scripts and physical presets

The previous approach that relied heavily on the model's full script generation showed signs of instability. The objective of this new approach is therefore to reduce the role of LLMs in code generation.
Instead of generating a complete script for all objects, we use a structure based on reusable scripts and predefined physical presets. All the interactive objects share common characteristics that I was able to group in a C# script (see Appendix). This includes generic parameters like stiffness, damping or response intensity. These common components allow consistent behavior between objects, more stable simulation and better runtime performance.
Then, the objects are classified into physical families. Each family has a specific behavior script and physical parameters preset.
A preset is a file that defines a set of physical parameters. They define the common physical parameters for each family such as density, friction, bounciness, etc.

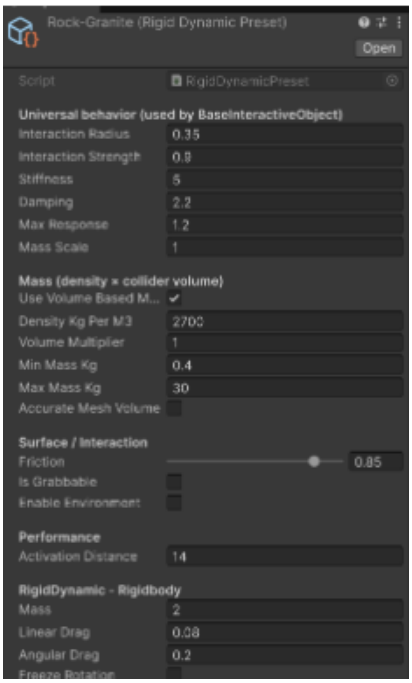


Figure 16: Example of a physical preset used to define common parameters such as density, friction and damping.

Objects can be dynamic rigid objects like stones, static objects like walls, wind-sensitive objects like small plants, structured vegetation like trees and others.
The use of families allows us to define environmental behaviors and allows us to obtain static objects in a scene or objects that can react to forces such as the wind.
For example, several types of rocks can be defined with different densities: heavy rock (high density, difficult to move), classical rock (balanced behavior), light rock (easier to move).

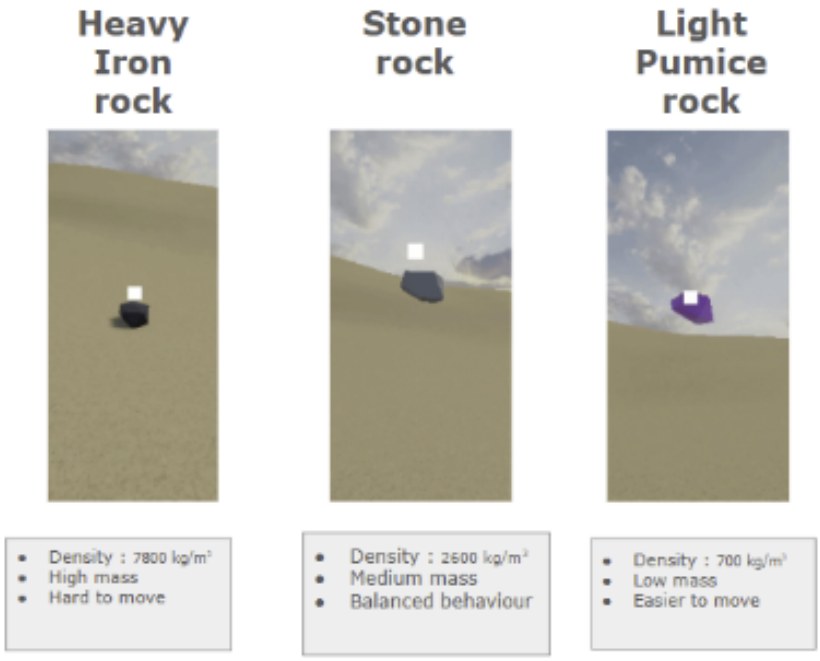


Figure 17: Examples of different physical behaviors obtained by applying distinct density presets to rock objects belonging to the same family.

The mass of objects can be calculated automatically. We use the formula: mass = density * volume. The volume is estimated by calculating the sum of the volumes of the colliders. When the colliders are well adapted to the mesh, the volume is well estimated.

In this architecture, LLMs are no longer responsible for generating the full code, their role is more limited. First, it identifies the object type from the mesh and an image. Then, it chooses the most appropriate physical preset for the object.

This method showed more stable results than previous methods. Using the reusable scripts and presets, I set up a small virtual environment containing different types of simple objects like rocks, bushes and trees, each using physical presets adapted to their family.

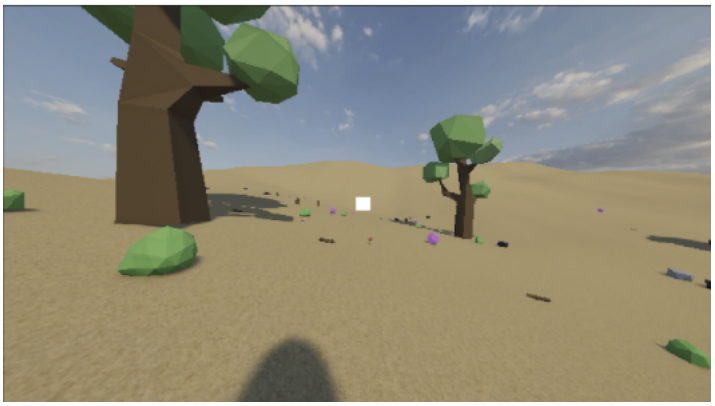


Figure 18: Example of a small virtual environment composed of objects configured using reusable scripts and physical presets.

However, this approach remains partially manual. Indeed, this method assumes that the colliders are already correctly defined. Thus, this approach is a good solution but does not yet constitute a fully autonomous solution.

4.4 AI agent for iterative physical authoring

In order to automate this process, the purpose of this approach is to create an AI agent capable of automatically improving physical configurations. Professor Xi Wang helped me with the design of this agent. The objective of this agent is to iteratively propose a physical configuration and evaluate its consistency to propose improvements until obtaining a satisfactory result. This agent should allow us to converge towards a coherent configuration without human intervention.

As part of this method, the agent is coded in Python and the model used is Qwen2.5-VL-Instruct.

The model is a Vision-Language model. It is a model that can analyze images.

The agent takes for input a mesh file (.obj) as well as any information about the object. The mesh is first loaded, cleaned and rendered from multiple viewpoints using PyRender. These renderings are then sent to the Vision-Language model which proposes a physical configuration (colliders + physical parameters). This configuration is converted into a JSON contract which is then judged by the model. In order to analyze the results, the agent returns a HTML report gathering all the information.

A JSON contract looks like this:

▼ Show contract JSON


```
{
  "colliders": [
    {
      "type": "box",
      "localPosition": [
        0.0,
        0.0,
        0.0
      ],
      "localRotation": [
        0.0,
        0.0,
        0.0
      ],
      "size": [
        0.855566,
        0.534573,
        0.873409
      ],
      "isTrigger": false
    }
  ],
  "physics": {
    "mass": 1.0,
    "useGravity": true,
    "isKinematic": false,
    "linearDamping": 0.05,
    "angularDamping": 0.05,
    "staticFriction": 0.6,
    "dynamicFriction": 0.6,
    "restitution": 0.05
  }
}
```


Figure 19: Example of a JSON contract.

In Unity, a C# script can read the JSON contract and instantiate the colliders and apply the physical parameters.

In each iteration loop, the model is called twice, once as a proposer and once as a judge. The judge analyzes the generated configuration and evaluates the quality of the colliders, their position and their consistency with the mesh. The model provides an explanation of the problem and a decision (continue or stop) with a level of confidence.
The agent's scripts are provided in the Appendix. Here is a diagram of the agent's architecture.

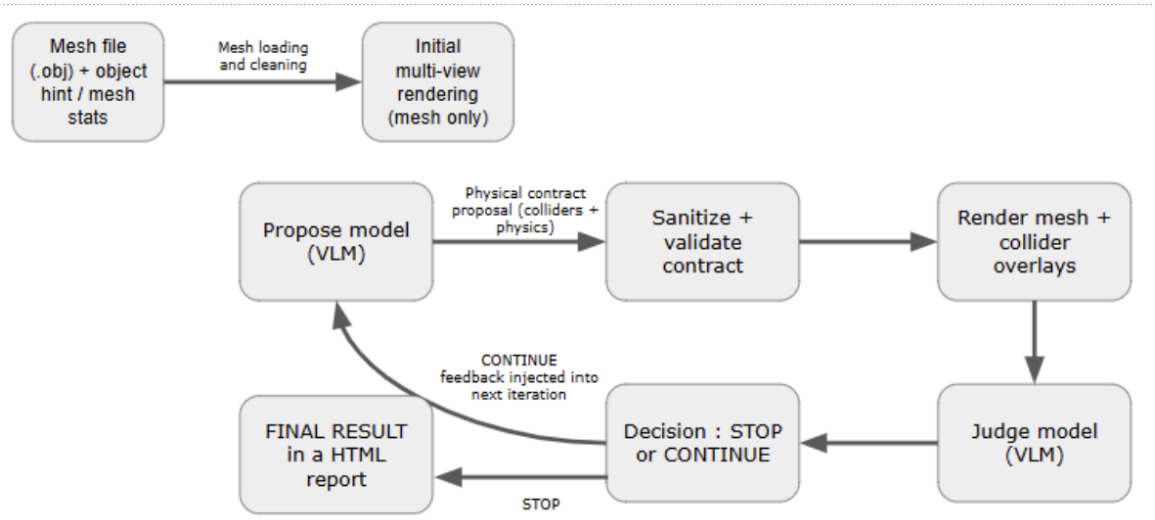


Figure 20: Architecture of the iterative AI agent.

The experiments showed that the agent can produce several iterations and can produce correct JSON contracts.

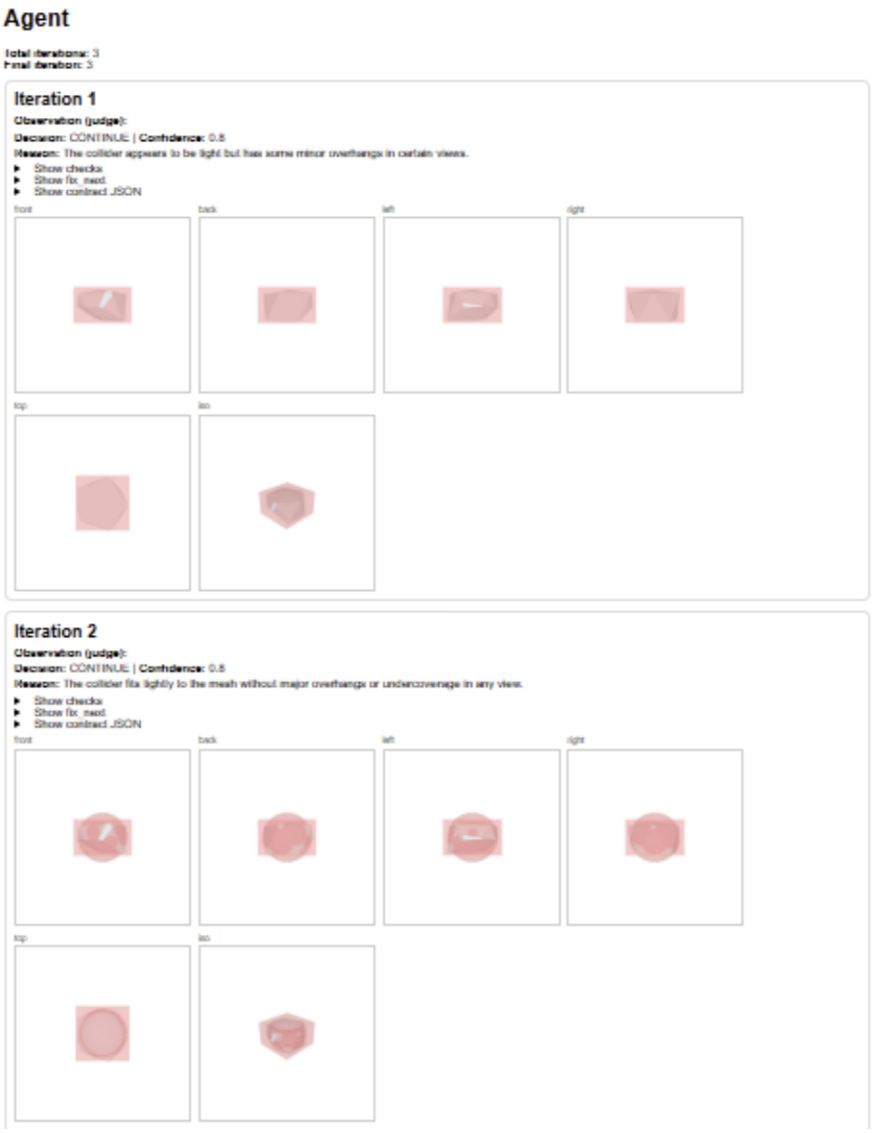


Figure 21: Example of output given by the agent.

The model can detect geometric inconsistencies and propose gradual improvements.



Figure 22: Example of a corrective suggestion produced by the agent to improve collider geometry.

However, the experiments were done on a personal computer with a local version of the model and this may have limited the performance of the agent. The iterations were slow, which reduced the number of iterations tested.
As a result, strong convergence could not be clearly observed.

5 Results

In this section, I present the results obtained with the different methods explored during the project. The objective is to compare approaches and their ability to generate coherent physical configurations in Unity. In summary, we studied the manual configuration in Unity, the direct generation of scripts with an LLM, the use of a structured prompts template, the use of reusable scripts with physical presets, and the iterative AI agent.

To compare the methods, we use a qualitative evaluation based on several criteria. Each method is evaluated according to these criteria:

- **Human effort**: the amount of manual work required to obtain the final configuration.
- **Runtime stability**: the stability of the simulation during execution.
- **Physical plausibility**: whether the object behaves in a physically credible way in the simulation.
- **Collider quality**: for methods that automatically create the collider, how accurately the collider matches the shape of the mesh.

The objective is not to produce a quantitative assessment but rather to observe the strengths and limitations of each approach.

5.1 Manual Unity setup

Obviously, if we manually configure the physical properties of objects, we get better results. Objects have realistic and precise physical attributes and colliders. The major problem lies in the amount of manual work required.
This method therefore serves as a qualitative reference to compare the other methods.

5.2 Direct LLM generation

Generating C# scripts directly with an LLM allows a much faster final result. From a mesh and some information, the model can produce a C# script containing colliders, a Rigidbody and some physical parameters.
The experiments showed that the model correctly understands the nature of the object and proposes plausible parameters but not standardized.
However, the generation of colliders is limited. The model can generate mesh colliders, but if we want simpler colliders, the generation is often inconsistent and unpredictable.
In addition, the generated code may contain compilation errors and depends heavily on the prompt formulation which makes the method unstable.

5.3 Structured prompt template

Using a structured prompt template improves the consistency of results. This approach allows for more coherent colliders and better placement of physical components.
However, this method remains time-consuming. Generation requires several interactions with the model and can be a lengthy process for each object.

5.4 Object family scripts and physical presets

This method allows for more stable and controllable results. It does not handle the generation of colliders but produces coherent physical characteristics according to the family to which the object belongs.
This approach assumes preliminary manual work to write the C# scripts and assumes that the colliders are already defined.

5.5 Iterative AI agent

The agent introduces a more automated approach based on an iterative process. In view of the limited experiments carried out, I believe that the results remain limited. However, I was able to observe that the model produces a syntactically correct JSON contract and offers consistent improvements for subsequent iterations. This approach shows the potential of a system capable of automatically improving physical configurations.

6 Future works

Given the limited time of the internship, several avenues for improvement could not be thoroughly explored and may be good avenues for future work.

The first approach would be to compare several models to compare their performance and thus deduce which one would be most useful in this situation.

Then, another direction would be to improve the iterative AI agent by using a more efficient and powerful machine to be able to do more iterations and obtain more significant results. In addition, it would be interesting to use more recent versions of the model to obtain more relevant answers.

Finally, we could also seek to improve the design of the iterative agent so that it can automatically generate a skeleton structure for certain objects such as trees. We could add guidelines and constraints to the model so that it can be guided. For example, the generated skeleton could be constrained to stay centered along the main axis of the object. A joint could also be placed at the ground contact point to define a stable base for the structure. For a tree, additional joints could be positioned along the branches to obtain flexible deformations. The number of joints could be defined proportionally to the length of the branches, to have finer articulations. These instructions could help the model generate coherent and adapted skeletons.

7 Conclusion

Finally, this project explored the potential of using generative AI to assist in the generation of physically-based scenes in Unity. The objective of this work was to investigate how effectively models can support developers in the process of generating colliders and configuring physical characteristics.

Traditionally, configuring the behaviour of physical objects is typically a manual task. Advances in generative AI and code generation have allowed us to explore several approaches and evaluate them qualitatively.

Using LLMs directly to generate C# scripts appeared as a misleading solution. It allows obtaining scripts very quickly but has revealed restrictive limits. The chosen parameters are often arbitrary and non-standardized and may require additional manual adjustments similar to those of the traditional method.

Structuring the prompts sent to the LLM helped stabilize and improving the consistency of the results. However, this method is time-consuming because it is based on multiple interactions with the LLM.

A partially manual solution consisted of dividing the objects into families and writing reusable scripts. Instead of generating complete scripts, the models simply find the appropriate parameters for the objects. This hybrid solution significantly improved the stability and consistency of the results, which allowed us to generate a small virtual world with simple objects.

Moreover, the iterative AI agent represents a first attempt to automate the entire process. This approach demonstrates the potential of a system capable of analyzing the structure of objects and progressively improving their physical configuration.

Thus, this project shows that LLMs can be a useful tool for assisting developers in configuring physical objects in Unity. The results suggest that hybrid approaches currently provide more robust solutions. Finally, approaches such as the AI agent show significant potential and could eventually enable the development of systems capable of automatically configuring interactive virtual environments in a more reliable and autonomous manner.

8 References

[1] Ilya Baran and Jovan Popović. Automatic rigging and animation of 3d characters. *ACM Transactions on graphics (TOG)*, 26(3):72–es, 2007.

[2] Kiran Nasim and Young J Kim. Physics-based assistive grasping for robust object manipulation in virtual reality. *Computer Animation and Virtual Worlds*, 29(3-4):e1820, 2018.

[3] Pau Vivas Zamora. Ai-assisted game development: designing and prototyping core gameplay systems. 2025.

[4] Zhan Xu, Yang Zhou, Evangelos Kalogerakis, Chris Landreth, and Karan Singh. Rignet: Neural rigging for articulated characters. *arXiv preprint arXiv:2005.00559*, 2020.

[5] Shaofeng Yin, Jiaxin Ge, Zora Zhiruo Wang, Xiuyu Li, Michael J Black, Trevor Darrell, Angjoo Kanazawa, and Haiwen Feng. Vision-as-inverse-graphics agent via interleaved multimodal reasoning. *arXiv preprint arXiv:2601.11109*, 2026.

A Appendix

A.1 Example of generated Unity script



```
using UnityEngine;

[RequireComponent(typeof(MeshFilter))]
public class WoodStumpPhysics : MonoBehaviour
{
    [Header("Mode")]
    public bool generateOnStart = true;
    public bool isStaticStump = false;   // True if this stump should not move
    public bool useConvexMeshCollider = true;

    [Header("Wood Physics")]
    [Tooltip("Approximate dry wood density in kg/m^3. Typical range: 500–900")]
    public float woodDensity = 700f;

    [Tooltip("Clamp the final mass so it stays practical for Unity physics")]
    public float minMass = 8f;
    public float maxMass = 80f;

    [Header("Drag")]
    public float linearDrag = 0.15f;
    public float angularDrag = 1.5f;

    [Header("Friction / Bounce")]
    [Range(0f, 1f)] public float dynamicFriction = 0.65f;
    [Range(0f, 1f)] public float staticFriction = 0.75f;
    [Range(0f, 1f)] public float bounciness = 0.02f;

    [Header("Stability")]
    [Tooltip("Lower center of mass makes the stump harder to tip over")]
    [Range(-1f, 1f)] public float centerOfMassYOffset = -0.15f;

    private void Start()
    {
        if (generateOnStart)
            SetupPhysics();
    }

    [ContextMenu("Setup Physics")]
    public void SetupPhysics()
    {
        MeshFilter mf = GetComponent<MeshFilter>();
        if (mf == null || mf.sharedMesh == null)
        {
            Debug.LogError("WoodStumpPhysics: No MeshFilter or mesh found.", this);
            return;
        }

        Mesh mesh = mf.sharedMesh;

        // Remove existing colliders if you want a clean rebuild
        Collider[] oldColliders = GetComponents<Collider>();
        for (int i = 0; i < oldColliders.Length; i++)
        {
            DestroyImmediate(oldColliders[i]);
        }

        // Create collider
        MeshCollider meshCollider = gameObject.AddComponent<MeshCollider>();
        meshCollider.sharedMesh = mesh;
        meshCollider.convex = useConvexMeshCollider && !isStaticStump;

        // Create and assign wood-like physic material
        PhysicsMaterial woodMat = new PhysicsMaterial("WoodStumpMaterial");
        woodMat.dynamicFriction = dynamicFriction;
        woodMat.staticFriction = staticFriction;
        woodMat.bounciness = bounciness;
        woodMat.frictionCombine = PhysicsMaterialCombine.Average;
        woodMat.bounceCombine = PhysicsMaterialCombine.Minimum;

        meshCollider.material = woodMat;

        // Rigidbody setup
        Rigidbody rb = GetComponent<Rigidbody>();
        if (rb == null)
            rb = gameObject.AddComponent<Rigidbody>();

        if (isStaticStump)
        {
            rb.isKinematic = true;
            rb.useGravity = false;
        }
        else
        {
            rb.isKinematic = false;
            rb.useGravity = true;

            // Estimate volume from renderer bounds as an approximation
            Bounds bounds = GetWorldOrLocalBounds();
            float approxVolume = bounds.size.x * bounds.size.y * bounds.size.z * 0.55f;
            // 0.55 factor because the stump does not fill the whole bounds box

            float estimatedMass = approxVolume * woodDensity;
            rb.mass = Mathf.Clamp(estimatedMass, minMass, maxMass);

            rb.linearDamping = linearDrag;   // Unity 6+
            rb.angularDamping = angularDrag; // Unity 6+

            // For older Unity versions, replace with:
            // rb.drag = linearDrag;
            // rb.angularDrag = angularDrag;

            rb.interpolation = RigidbodyInterpolation.Interpolate;
            rb.collisionDetectionMode = CollisionDetectionMode.ContinuousDynamic;

            // Lower center of mass a bit so it feels like a stump
            Vector3 localCenter = mesh.bounds.center;
            localCenter.y += mesh.bounds.size.y * centerOfMassYOffset;
            rb.centerOfMass = localCenter;
        }
    }

    private Bounds GetWorldOrLocalBounds()
    {
        Renderer rend = GetComponent<Renderer>();
        if (rend != null)
            return rend.bounds;

        MeshFilter mf = GetComponent<MeshFilter>();
        return mf.sharedMesh.bounds;
    }
}
```

A.2 Prompt template for guided physical authoring

The following prompt template was used to guide the structured specification and generation of physically plausible Unity runtime configurations for 3D objects.

```
Prompt 1 --- Context & Conversation Setup

You are assisting with the physical authoring of a Unity prefab.

Your task is to help design a physically plausible runtime representation for a
    3D object used in a Unity scene.

The goal is NOT physically accurate simulation.
The goal is to create behaviour that is:

- perceptually plausible
- stable during runtime
- performant
- suitable for gameplay interaction

Technical constraints:

- runtime only
- no Unity Editor scripts
- no external packages
- use only built-in Unity components

Allowed components include:

- Colliders
- Rigidbody
- Joints
- MonoBehaviour scripts
- MaterialPropertyBlock

Design principles:

1. Modularity
Enable only the behaviour systems required by the object.

2. Visual / Physics separation
Visual motion must never destabilize physics.
Physics colliders must remain stable.

3. Proxy physics first
Prefer primitive colliders over MeshColliders.

4. Collider budget control
Avoid unnecessarily complex collider setups.

5. Minimal complexity
Only introduce behaviour systems if justified by the object's role.

Interaction model for this conversation:

You must NOT assume information about the object.
```

```
Instead:

1. Ask questions about the asset.
2. Wait for the user to provide information.
3. Use that information to progressively build a physical specification.

The user may provide:

- mesh previews
- screenshots
- Unity hierarchy
- inspector views
- descriptions of gameplay usage

Do not finalize decisions until enough information is available.

Start by asking questions required for object identification.


Prompt 2 --- Object Identification Questions

We are starting the specification process.

Step 1: Object Identification

Your task is to identify the object at a perceptual and functional level.

Ask a short questionnaire to obtain the following information:

- object name
- visual description
- object family or type
- intended role in the scene
- approximate scale relative to the player
- expected physical impression ("must feel like")

Examples of useful questions:

- What is the name of the object?
- Can you show screenshots of the mesh?
- What role does the object play in the scene?
- Is the object small, medium, or large compared to the player?
- How should the object feel physically? (heavy, rigid, fragile, flexible, etc.)

Ask the questions first and wait for the user to respond.

Do not produce the final identification summary yet.

Once the user has answered, you may send:

Using the information provided, produce a structured summary titled:

Object Identification
```

```
Prompt 3 --- Structural & Asset Analysis

Step 2: Structural & Asset Analysis

Your task is to analyze the structure of the asset.

Ask targeted questions to determine:

Mesh structure

- Is the mesh monolithic or composed of multiple meshes?
- Are there child meshes or submeshes?
- Are there LOD levels?

Materials

- Is there a single material or multiple materials?
- Do materials correspond to distinct regions?

Unity hierarchy

If available, request:

- root object name
- child objects
- location of MeshRenderers / MeshFilters
- presence of LODGroup

Existing components

Ask whether the object already contains:

- colliders
- rigidbody
- joints

Inference limits

You must acknowledge:

What can be inferred from the mesh:
- general bounding volume
- dominant orientation
- approximate shape

What cannot be reliably inferred:
- semantic parts
- articulation points
- material properties

Ask the required questions and wait for the user's answers.

After receiving the information, produce a block titled:
```

```
Asset Structure Summary


Prompt 4 --- Physical Classification

Step 3: Physical Category + Part Complexity

Using the information gathered so far, classify the object.

Select ONE physical category:

- Fully rigid object
- Mostly rigid object with slight movement
- Flexible vegetation
- Decorative non-interactive object

Select ONE part complexity level:

- Single-part
- Multi-part

Multi-part means different regions of the object may require different
    behaviours.

Provide a short justification (2--4 sentences).

Output block:

Physical Category + Part Complexity


Prompt 5 --- Module Selection

Step 4: Module Selection

Determine which behaviour modules should be enabled.

Available modules:

ProxyColliderModule
RigidInteractionModule
Environmental Visual Motion (EVM)
ArticulationModule
DeformationModule

For each module output:

ModuleName: ON/OFF --- justification

Only enable modules if clearly required.

Output block:

Module Selection
```

```
Prompt 6 --- Collider Authoring Plan

Step 5: Collider Authoring Plan

Your task is to design the collider system.

If needed, ask additional questions about:

- interaction distance
- collision precision requirements
- raycast targeting
- collider performance budget
- important structural regions (trunk, branches, tips, etc.)

Choose ONE collider strategy:

- Single primitive collider
- Compound primitive colliders
- Structured multi-part proxy colliders
- High-fidelity proxy colliders

Then produce a block titled:

Collider Authoring Plan

The plan must include:

Collider hierarchy

Example structure:

Root
|-- Visual
\-- Physics
    \-- Colliders

Collider groups

For each collider:

- name
- collider type
- role
- placement guideline
- approximate size

Raycast rules

Which colliders should respond to raycasts.

Collider budget

- expected collider range
- hard maximum collider count
```

```
- degradation strategy


Prompt 7 --- Interaction Intent

Step 6: Interaction Intent

Determine how the player interacts with the object.

Ask questions to determine:

Interaction mode (choose one):

- None
- Push-only
- Drag-at-point
- Pickup / Hold

Movement properties:

- movable or static
- resistance level

Grab behaviour:

- grabbable or not

Interaction constraints:

- interaction distance
- release conditions
- layer filtering

After receiving the answers, produce a block titled:

Interaction Intent


Prompt 8 --- Deformation & Motion

Step 7: Deformation & Motion Intent

Determine whether the object requires deformation or articulated motion.

Choose exactly one:

- No deformation
- Visual-only deformation
- Articulated rigid parts
- Rigged deformation
- Softbody / experimental

Provide a short justification explaining the choice.

Output block:
```

```
Deformation & Motion Intent


Prompt 9 --- Parameter Set

Step 8: Physical Parameter Set

Define a minimal set of parameters controlling the object's behaviour.

Constraints:

- 5 to 10 parameters maximum
- each parameter must be float or bool
- parameters correspond to public fields in Unity scripts

Parameters should influence:

- interaction feel
- motion behaviour
- collider stability
- contact realism

For each parameter include:

- name
- meaning
- typical range
- default value
- modules using it

Also list any implementation fields separately.

Output block:

Parameter Set


Prompt 10 --- Unity Implementation Strategy

Step 9: Unity Implementation Strategy

Describe how the system should be implemented in Unity.

Do NOT write code.

Provide:

Prefab hierarchy

Example:

Root
|-- Visual
\-- Physics
```

```
    \-- GeneratedColliders

Components per node

Explain which components belong on:

- Root
- Visual
- Physics

Module mapping

Explain which scripts implement each module.

Stability recommendations

Examples:

- Rigidbody interpolation
- collision detection mode
- conservative drag values

Output block:

Unity Implementation Strategy


Prompt 11 --- Code Generation

Once the full discussion is finished:

We now enter the code generation phase.

Below is the full specification produced during the conversation.

[INSERT SPECIFICATION HERE]

Generate Unity runtime code implementing this specification.

Constraints:

- runtime only
- no Unity Editor scripts
- no external packages
- Unity built-in components only

Requirements:

- maintain Visual / Physics separation
- stable physics behaviour
- conservative forces
- collider generation only once during initialization
- readable and well-structured code

Choose the simplest architecture that satisfies the specification:
```

```
Possible structures:

Simple objects
1 script

Moderate complexity
ObjectProxySetup + ObjectBehavior

Articulated objects
ObjectProxySetup + ObjectBehavior + ObjectArticulation

Provide:

1. Complete C# scripts
2. Instructions for attaching the scripts in Unity
3. Required prefab hierarchy
4. Recommended Rigidbody settings
```

A.3 AI Agent Implementation

The iterative physical authoring agent was implemented in Python as a modular pipeline. Its role is to analyze a mesh, render multiple views, call a vision-language model to propose a physical contract, evaluate the result through a judge phase, and iteratively improve the generated collider configuration. The implementation is divided into several files handling the main loop, mesh loading, prompting, rendering, reporting, model interaction, and schema validation. The execution loop follows a propose–judge strategy with contract sanitization and validation before each new iteration. :contentReference[oaicite:0]index=0 The entry point loads the mesh, initializes the client, launches the loop, saves the resulting JSON file, and generates an HTML report summarizing all iterations. :contentReference[oaicite:1]index=1

A.3.1 Main execution file (`main.py`)

Listing 1: Main entry point of the iterative agent.

```python
from __future__ import annotations

import argparse
import json
import webbrowser
from datetime import datetime
from pathlib import Path

from mesh_read import load_mesh_for_judge, mesh_quick_stats
from model_client import ModelClient
from loop import run_loop
from render_pyrender import render_multiview
from report import write_final_report


def main():
    #arguments parsing
    ap = argparse.ArgumentParser()
    #path to the mesh
```

```
 ap.add_argument("--mesh", required=True, help="Path to mesh file (OBJ
recommended)")
 #max iterations
 ap.add_argument("--max-iters", type=int, default=6)
 #backend mode (stub or transformers)
 ap.add_argument("--mode", type=str, default="transformers", help="stub |
transformers")
 #out : name of JSON file
 ap.add_argument("--out", type=str, default="result.json")
 #img size (square)
 ap.add_argument("--img-size", type=int, default=384)
 #optionnal text (ex: "rock") to guide model
 ap.add_argument("--obj-hint", type=str, default=None, help="Optional text
hint: 'rock', 'tree', etc.")
 args = ap.parse_args()

 #load mesh
 mesh = load_mesh_for_judge(args.mesh)
 stats = mesh_quick_stats(mesh)

 stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
 run_dir = Path("runs") / stamp
 run_dir.mkdir(parents=True, exist_ok=True)
 #cleint initialization
 client = ModelClient(mode=args.mode, debug_dir = run_dir / "raw")

 #wrapper
 def render_fn(m, out_size=(512, 512), contract=None,
overlay_colliders=False):
     return render_multiview(m,out_size=out_size, contract=contract,
overlay_colliders=overlay_colliders)

 #run loop
 result = run_loop(
     mesh=mesh,
     client=client,
     render_fn=render_fn,
     max_iters=args.max_iters,
     out_size=(args.img_size, args.img_size),
     obj_hint=args.obj_hint or stats,
     run_dir=run_dir
 )

 print("Total iterations:", len(result["history"]))

 #save JSON
 (run_dir / args.out).write_text(json.dumps(result, indent=2),
encoding="utf-8")
 print("Saved:", run_dir / args.out)

 #generate report
 report_path = write_final_report(run_dir, result)  # <-- NEW
 print("Wrote report:", report_path)

 try:
```

```
        webbrowser.open(report_path.resolve().as_uri())
    except Exception:
        pass


if __name__ == "__main__":
    main()
```

A.3.2 Iteration loop (`loop.py`)

Listing 2: Main propose–judge loop of the agent.

```
from __future__ import annotations #for annotations
from typing import Dict, Any, List, Optional, Callable
from pathlib import Path #manipulations of paths

from schema import validate_contract_dict
from prompting import build_propose_prompt_vlm, build_judge_prompt_vlm
from report import save_multiview_images

#correct a contract that can be false
def sanitize_contract_inplace(contract: Dict[str, Any]) -> None:
    #check if colliders exist
    cols = contract.get("colliders", [])
    if not isinstance(cols, list):
        return

    for c in cols:
        if not isinstance(c, dict):
            continue
        t = str(c.get("type", "box")).lower()
        size = c.get("size", [1.0, 1.0, 1.0])

        #ensure size is list of 3 numbers
        if not isinstance(size, list) or len(size) != 3:
            size = [1.0, 1.0, 1.0]

        #sphere: size[0] radius must be > 0, size[1]=size[2]=0
        if t == "sphere":
            r = float(size[0]) if isinstance(size[0], (int, float)) else 0.0
            if r <= 0:
                r = 0.25  # safe default radius
            c["size"] = [r, 0.0, 0.0]

        #capsule: size[0]=radius >0, size[1]=height >0, size[2]=0
        elif t == "capsule":
            r = float(size[0]) if isinstance(size[0], (int, float)) else 0.0
            h = float(size[1]) if isinstance(size[1], (int, float)) else 0.0
            if r <= 0:
                r = 0.2
            if h <= 0:
                h = max(2 * r, 0.5)
            c["size"] = [r, h, 0.0]
```

```
        #box: all >0
        elif t == "box":
            x = float(size[0]) if isinstance(size[0], (int, float)) else 0.0
            y = float(size[1]) if isinstance(size[1], (int, float)) else 0.0
            z = float(size[2]) if isinstance(size[2], (int, float)) else 0.0
            if x <= 0: x = 0.5
            if y <= 0: y = 0.5
            if z <= 0: z = 0.5
            c["size"] = [x, y, z]


# propose --> sanitize + validate --> render overlay --> judge --> feedback -->
    stop or continue ?
def run_loop(
    mesh,
    client,
    render_fn: Callable[..., Dict[str, Any]],
    max_iters: int = 6,
    out_size=(512, 512),
    obj_hint: Optional[str] = None,
    run_dir: Optional[Path] = None,
) -> Dict[str, Any]:
    history: List[Dict[str, Any]] = []
    last_decision: Optional[Dict[str, Any]] = None
    last_contract: Optional[Dict[str, Any]] = None
    best_guess: Optional[Dict[str, Any]] = None

    #0) PRE-RENDER mesh-only views ONCE (same for all iterations)
    propose_images_cached = render_fn(
        mesh,
        out_size=out_size,
        contract=None,
        overlay_colliders=False,
    )

    for it in range(1, max_iters + 1):
        #1) PROPOSE phase: reuse cached mesh-only images
        propose_images = propose_images_cached

        propose_prompt = build_propose_prompt_vlm(
            iteration=it,
            previous_contract=last_contract,
            judge_feedback=last_decision,
            history_brief=history[-2:],
        )

        if obj_hint:
            propose_prompt = f"OBJECT_HINT:\n{obj_hint}\n\n" + propose_prompt

        contract = client.propose(images=propose_images, prompt=propose_prompt)

        sanitize_contract_inplace(contract)
        validate_contract_dict(contract)

        #2) JUDGE phase: render overlay (must be re-rendered each iter)
```

```
        judge_images = render_fn(
            mesh,
            out_size=out_size,
            contract=contract,
            overlay_colliders=True,
        )

        judge_prompt = build_judge_prompt_vlm(iteration=it)
        decision = client.judge(images=judge_images, prompt=judge_prompt)

        #normalize fix_next
        fix_next = decision.get("fix_next", [])
        if isinstance(fix_next, dict):
            fix_next = fix_next.get("actionable_visual_fixes", [])
        if fix_next is None:
            fix_next = []
        decision["fix_next"] = fix_next

        step = {
            "iteration": it,
            "contract": contract,
            "decision": decision,
        }
        history.append(step)

        #save images for this iteration
        if run_dir is not None:
            iter_dir = Path(run_dir) / "iters" / f"iter_{it:02d}"
            save_multiview_images(mesh, contract, iter_dir, out_size=out_size)

        last_contract = contract
        last_decision = decision
        best_guess = {"contract": contract, "iteration": it, "decision":
    decision}

        checks = decision.get("checks", {})
        checks_ok = (
            isinstance(checks, dict)
            and len(checks) > 0
            and all(bool(v) for v in checks.values())
        )

        if str(decision.get("decision", "")).upper() == "STOP" and checks_ok:
            return {"final": best_guess, "best_guess": best_guess, "history":
    history}

     return {"final": None, "best_guess": best_guess, "history": history}
```

A.3.3 Mesh loading and preprocessing (`mesh_read.py`)

Listing 3: Utilities for loading and summarizing mesh files.

```
from __future__ import annotations
from typing import Optional
```

```
import trimesh


#load mesh from file, clean it to avoid problems
def load_mesh_for_judge(mesh_path: str) -> trimesh.Trimesh:
    m = trimesh.load(mesh_path, force="mesh")

    if isinstance(m, trimesh.Scene):
        m = trimesh.util.concatenate(tuple(m.geometry.values()))

    # Clean up
    try:
        m.remove_unreferenced_vertices()
    except Exception:
        pass

    try:
        m.remove_degenerate_faces()
    except Exception:
        if hasattr(m, "nondegenerate_faces"):
            try:
                m.update_faces(m.nondegenerate_faces())
            except Exception:
                pass

    try:
        m.remove_unreferenced_vertices()
    except Exception:
        pass

    return m

#read text from OBJ file without loading all the file in prompt
def read_obj_text_excerpt(mesh_path: str, max_chars: int = 20000) -> str:
    try:
        with open(mesh_path, "r", encoding="utf-8", errors="ignore") as f:
            text = f.read(max_chars)
    except Exception:
        return ""
    if len(text) >= max_chars:
        text += "\n\n# [TRUNCATED OBJ TEXT]\n"
    return text


def mesh_quick_stats(m: trimesh.Trimesh) -> str:
    """
    small text summary you can pass to the VLM (safe + useful)
    """
    try:
        ext = m.extents
        bb_min, bb_max = m.bounds
        return (
            f"vertices={len(m.vertices)}, faces={len(m.faces)}, "
            f"extents={ext.tolist()}, "
            f"bounds_min={bb_min.tolist()}, bounds_max={bb_max.tolist()}, "
```

```
            f"watertight={bool(getattr(m, 'is_watertight', False))}"
        )
    except Exception:
        return "mesh_stats_unavailable"
```

A.3.4 Prompt construction (`prompting.py`)

Listing 4: Prompt generation for the propose and judge phases.

```
from __future__ import annotations
import json
from typing import Dict, Any, Optional, List


CONTRACT_SCHEMA = {
    "colliders": [
        {
            "type": "box|sphere|capsule",
            "localPosition": [0, 0, 0],
            "localRotation": [0, 0, 0],
            "size": [0, 0, 0],
            "isTrigger": False,
        }
    ],
    "physics": {
        "mass": 1.0,
        "useGravity": True,
        "isKinematic": False,
        "linearDamping": 0.05,
        "angularDamping": 0.05,
        "staticFriction": 0.6,
        "dynamicFriction": 0.6,
        "restitution": 0.05,
    },
}

#prompt for propose phase
def build_propose_prompt_vlm(
    iteration: int,
    previous_contract: Optional[Dict[str, Any]] = None,
    judge_feedback: Optional[Dict[str, Any]] = None,
    history_brief: Optional[List[Dict[str, Any]]] = None,
) -> str:

    output_template = {
        "colliders": [
            {
                "type": "box",
                "localPosition": [0.0, 0.0, 0.0],
                "localRotation": [0.0, 0.0, 0.0],
                "size": [1.0, 1.0, 1.0],
                "isTrigger": False,
            }
        ],
```

```
        "physics": {
            "mass": 1.0,
            "useGravity": True,
            "isKinematic": False,
            "linearDamping": 0.05,
            "angularDamping": 0.05,
            "staticFriction": 0.6,
            "dynamicFriction": 0.6,
            "restitution": 0.05,
        },
    }

    constraints = [
        "Return ONLY a single JSON object.",
        "Output MUST be on a single line (minified).",
        "Use double quotes for all keys and string values.",
        "No markdown. No backticks. No comments. No extra text.",
        "No trailing commas.",
        "Top-level JSON keys MUST be exactly: colliders, physics.",
        "You MUST follow EXACTLY the output_template structure (same keys, same
    nesting).",
        "You may ONLY change numeric values and the collider 'type' fields.",
        "You must NOT add keys, remove keys, or rename keys.",
        "Use 1 to 8 colliders (duplicate the collider object if you need
    more).",
        "Collider.type must be one of: box, sphere, capsule.",
        "Box size is full extents [x,y,z] and must be > 0.",
        "Sphere uses size[0]=radius, and set size[1]=size[2]=0.",
        "Capsule uses size[0]=radius and size[1]=total height (including caps),
    set size[2]=0.",
        "You may duplicate the collider object inside the 'colliders' array to
    create multiple colliders (1 to 8).",
        "If the object shape is irregular, you are encouraged to use 2-5
    colliders instead of one large collider."
    ]

    hist = history_brief[-2:] if history_brief else None

    payload = {
        "iteration": iteration,
        "task": "Fill the output_template to propose colliders + physics for
    the object shown in the images.",
        "output_template": output_template,
        "constraints": constraints,
        "previous_contract": previous_contract,
        "judge_feedback": judge_feedback,
        "recent_history_brief": hist,
        "notes": [
            "The images show the SAME object from different views.",
            "Colliders must visually match the object silhouette.",
            "If judge_feedback says CONTINUE, apply fix_next by changing
    numbers (size/position/rotation) or adding a collider.",
        ],
    }
```

```
    return (
        "You are a specialist in creating physics proxies (colliders) for Unity
    assets.\n"
        "You will be given multiple images of the SAME object from different
    views.\n"
        "Fill the JSON output_template. Return ONLY the JSON, on a single
    line.\n\n"
        f"CONTEXT_JSON:\n{json.dumps(payload, indent=2)}\n"
    )


#prompt for judge phase
def build_judge_prompt_vlm(iteration: int) -> str:
    schema = {
        "decision": "STOP or CONTINUE",
        "confidence": 0.0,
        "checks": {
            "tight_fit": True,
            "no_major_overhang": True,
            "no_major_undercoverage": True,
            "reasonable_count": True,
            "alignment_ok": True
        },
        "reason": "short visual reason",
        "fix_next": ["actionable visual fixes"]
    }

    payload = {
        "iteration": iteration,
        "task": "Judge if collider overlay is a TIGHT physics proxy for the
    mesh.",
        "strict_rules": [
            "Be strict. Prefer CONTINUE unless the fit is clearly tight.",
            "STOP only if all checks are true.",
            "CONTINUE if you see large empty red volume around the mesh in any
    view.",
            "CONTINUE if collider exceeds mesh silhouette noticeably in 2 or
    more views.",
            "CONTINUE if collider misses visible parts of the mesh
    (under-coverage).",
            "Prefer 2-5 colliders for irregular rocks instead of one huge box.",
            "IF decision is CONTINUE, fix_next MUST contain at least 2 concrete
    numeric edits.",
            "fix_next must NEVER be empty when CONTINUE.",
            "Each fix must mention: which collider index, what parameter
    (size/pos/rot), and an amount or percentage."
        ],
        "output_schema": schema
    }

    return (
        "You are a strict visual judge for collider fitting.\n"
        "You will be given images showing the mesh WITH collider overlays.\n"
        "Return ONLY valid JSON.\n\n"
        f"CONTEXT_JSON:\n{json.dumps(payload, indent=2)}\n"
```

```
    )
```

A.3.5 Model client (`model_client.py`)

Listing 5: Interface with the vision-language model and robust JSON parsing.

```
from __future__ import annotations

import json
import os
import re
from pathlib import Path
from typing import Any, Dict, List, Literal, Optional, Tuple

                        ![Result](img/res1.png)
import torch
from PIL import Image
from transformers import AutoProcessor, AutoModelForImageTextToText

TaskKind = Literal["propose", "judge"]


def _extract_json_objects(text: str) -> List[str]:
    text = text.strip()
    text = re.sub(r"^```json\s*", "", text, flags=re.IGNORECASE)
    text = re.sub(r"^```\s*", "", text)
    text = re.sub(r"\s*```$", "", text)

    objs: List[str] = []
    depth = 0
    start: Optional[int] = None
    in_str = False
    esc = False

    for i, ch in enumerate(text):
        if in_str:
            if esc:
                esc = False
            elif ch == "\\":
                esc = True
            elif ch == '"':
                in_str = False
            continue
        else:
            if ch == '"':
                in_str = True
                continue
            if ch == "{":
                if depth == 0:
                    start = i
                depth += 1
            elif ch == "}":
                if depth > 0:
                    depth -= 1
                    if depth == 0 and start is not None:
```

```
                        objs.append(text[start : i + 1])
                        start = None

    return objs


def _is_contract_dict(d: Any) -> bool:
    return isinstance(d, dict) and "colliders" in d and "physics" in d


def _parse_json_robust(text: str) -> Dict[str, Any]:
    objs = _extract_json_objects(text)

    parsed_candidates: List[Dict[str, Any]] = []
    for raw in objs:
        raw2 = re.sub(r",\s*([}\]])", r"\1", raw)
        try:
            parsed = json.loads(raw2)
            if isinstance(parsed, dict):
                parsed_candidates.append(parsed)
        except Exception:
            continue

    for d in reversed(parsed_candidates):
        if _is_contract_dict(d):
            return d

    if parsed_candidates:
        return parsed_candidates[-1]

    raise ValueError("No valid JSON object could be parsed from model output.")


class ModelClient:
    def __init__(
        self,
        mode: str = "transformers",
        model_id: Optional[str] = None,
        temperature: float = 0.2,
        max_image_size: int = 768,
        max_new_tokens_propose: int = 700,
        max_new_tokens_judge: int = 350,
        force_cpu: bool = False,
        debug_dir: Optional[Path] = None,
    ):
        self.mode = mode
        self.temperature = float(temperature)
        self.max_image_size = int(max_image_size)
        self.max_new_tokens_propose = int(max_new_tokens_propose)
        self.max_new_tokens_judge = int(max_new_tokens_judge)

        self.debug_dir = debug_dir
        self._call_index = 0

        if self.mode == "stub":
```

```
            self.model_id = None
            self.processor = None
            self.model = None
            return

        self.model_id = model_id or os.environ.get("QWEN_VLM_MODEL_ID",
"Qwen/Qwen2.5-VL-3B-Instruct")
        self.processor = AutoProcessor.from_pretrained(self.model_id,
trust_remote_code=True)

        dtype = torch.float16 if torch.cuda.is_available() and not force_cpu
else torch.float32
        device_map = "auto" if torch.cuda.is_available() and not force_cpu else
None

        self.model = AutoModelForImageTextToText.from_pretrained(
            self.model_id,
            torch_dtype=dtype,
            device_map=device_map,
            trust_remote_code=True,
        )
        self.model.eval()

    def _log_raw(self, kind: str, text: str) -> None:
        if self.debug_dir is None:
            return
        try:
            self.debug_dir.mkdir(parents=True, exist_ok=True)
            self._call_index += 1
            p = self.debug_dir / f"raw_{self._call_index:04d}_{kind}.txt"
            p.write_text(text, encoding="utf-8", errors="ignore")
        except Exception:
            pass

    def _resize_keep_aspect(self, img: Image.Image) -> Image.Image:
        img = img.convert("RGB")
        w, h = img.size
        m = max(w, h)
        if m <= self.max_image_size:
            return img
        scale = self.max_image_size / float(m)
        return img.resize((int(w * scale), int(h * scale)))

    def _ordered_views(self, images: Dict[str, Image.Image]) -> List[Tuple[str,
Image.Image]]:
        order = ["front", "back", "left", "right", "top", "iso"]
        ordered = [(k, images[k]) for k in order if k in images]
        return ordered if ordered else list(images.items())

    def call_model(self, images: Dict[str, Image.Image], prompt: str, kind:
TaskKind) -> str:
        if self.mode == "stub":
            if kind == "judge":
                out = json.dumps(
```

```
                {"decision": "CONTINUE", "confidence": 0.2, "reason":
"stub", "fix_next": ["implement"]}
            )
            self._log_raw(kind, out)
            return out

        out = json.dumps(
            {
                "colliders": [
                    {
                        "type": "box",
                        "localPosition": [0, 0, 0],
                        "localRotation": [0, 0, 0],
                        "size": [1, 1, 1],
                        "isTrigger": False,
                    }
                ],
                "physics": {
                    "mass": 1.0,
                    "useGravity": True,
                    "isKinematic": False,
                    "linearDamping": 0.05,
                    "angularDamping": 0.05,
                    "staticFriction": 0.6,
                    "dynamicFriction": 0.6,
                    "restitution": 0.05,
                },
            }
        )
        self._log_raw(kind, out)
        return out

    max_new = self.max_new_tokens_judge if kind == "judge" else
self.max_new_tokens_propose

    ordered = self._ordered_views(images)
    pil_imgs = [self._resize_keep_aspect(im) for _, im in ordered]
    view_names = [k for k, _ in ordered]

    message_content: List[Dict[str, Any]] = [{"type": "text", "text":
prompt}]
    for name, im in zip(view_names, pil_imgs):
        message_content.append({"type": "text", "text": f"View: {name}"})
        message_content.append({"type": "image", "image": im})

    messages = [{"role": "user", "content": message_content}]

    text = self.processor.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
    inputs = self.processor(text=[text], images=pil_imgs,
return_tensors="pt").to(self.model.device)

    if kind == "propose":
        do_sample = False
        temperature = None
```

```
        else:
            do_sample = self.temperature > 0
            temperature = self.temperature if self.temperature > 0 else None

        with torch.no_grad():
            generated_ids = self.model.generate(
                **inputs,
                max_new_tokens=max_new,
                do_sample=do_sample,
                temperature=temperature,
            )

        new_tokens = generated_ids[0][inputs["input_ids"].shape[1] :]
        out = self.processor.decode(new_tokens, skip_special_tokens=True)

        self._log_raw(kind, out)
        return out

    def propose(self, images: Dict[str, Image.Image], prompt: str) -> Dict[str,
Any]:
        raw = self.call_model(images, prompt, "propose")

        def must_be_contract(d: Dict[str, Any]) -> bool:
            return _is_contract_dict(d)

        try:
            parsed = _parse_json_robust(raw)
            if not must_be_contract(parsed):
                raise ValueError("Parsed JSON is not a contract (missing
colliders/physics).")
            return parsed
        except Exception:
            repair_base = (
                "Return ONLY one valid JSON object.\n"
                "Top-level keys MUST be exactly: \"colliders\" and
\"physics\".\n"
                "Do NOT wrap the result in another key (no \"contract\", no
\"result\").\n"
                "Use double quotes. No trailing commas. No comments. No extra
text.\n"
                "Follow the output_template structure exactly."
            )
            for _ in range(3):
                raw = self.call_model(images, repair_base +
"\nINVALID_OUTPUT:\n" + raw, "propose")
                try:
                    parsed = _parse_json_robust(raw)
                    if must_be_contract(parsed):
                        return parsed
                except Exception:
                    continue
            raise

    def judge(self, images: Dict[str, Image.Image], prompt: str) -> Dict[str,
Any]:
```

```
        raw = self.call_model(images, prompt, "judge")
        try:
            return _parse_json_robust(raw)
        except Exception:
            repair_base = (
                "Return ONLY one valid JSON object. "
                "Use double quotes. No trailing commas. No comments. No extra
    text."
            )
            for _ in range(3):
                raw = self.call_model(images, repair_base +
    "\nINVALID_OUTPUT:\n" + raw, "judge")
                try:
                    return _parse_json_robust(raw)
                except Exception:
                    continue
            raise
```

A.3.6 Rendering module (`render_pyrender.py`)

Listing 6: Multiview rendering of the mesh and collider overlays.

```
from __future__ import annotations

from typing import Dict, Any, Optional, Tuple

import numpy as np
import pyrender
import trimesh
from PIL import Image


def _euler_deg_to_rotmat(euler_deg: np.ndarray) -> np.ndarray:
    rx, ry, rz = np.deg2rad(euler_deg)
    cx, sx = np.cos(rx), np.sin(rx)
    cy, sy = np.cos(ry), np.sin(ry)
    cz, sz = np.cos(rz), np.sin(rz)

    Rx = np.array([[1, 0, 0], [0, cx, -sx], [0, sx, cx]])
    Ry = np.array([[cy, 0, sy], [0, 1, 0], [-sy, 0, cy]])
    Rz = np.array([[cz, -sz, 0], [sz, cz, 0], [0, 0, 1]])
    return Rz @ Ry @ Rx


def _camera_pose(eye, target=(0, 0, 0), up=(0, 1, 0)) -> np.ndarray:
    eye = np.array(eye, dtype=float)
    target = np.array(target, dtype=float)
    up = np.array(up, dtype=float)

    z = eye - target
    z = z / (np.linalg.norm(z) + 1e-9)
    x = np.cross(up, z)
    x = x / (np.linalg.norm(x) + 1e-9)
    y = np.cross(z, x)
```

```
    pose = np.eye(4)
    pose[:3, 0] = x
    pose[:3, 1] = y
    pose[:3, 2] = z
    pose[:3, 3] = eye
    return pose


def _make_col_trimesh(c: Dict[str, Any]) -> Optional[trimesh.Trimesh]:
    t = str(c.get("type", "box")).lower()
    size = np.array(c.get("size", [1, 1, 1]), dtype=float)

    if t == "box":
        return trimesh.creation.box(extents=[float(size[0]), float(size[1]),
    float(size[2])])

    if t == "sphere":
        return trimesh.creation.icosphere(subdivisions=2, radius=float(size[0]))

    if t == "capsule":
        r = float(size[0])
        h_total = float(size[1])
        cyl_h = max(0.0, h_total - 2.0 * r)
        return trimesh.creation.capsule(radius=r, height=cyl_h)

    return None


def render_multiview(
    mesh: trimesh.Trimesh,
    out_size: Tuple[int, int] = (768, 768),
    contract: Optional[Dict[str, Any]] = None,
    overlay_colliders: bool = False,
) -> Dict[str, Image.Image]:
    m = mesh.copy()
    center = m.centroid.copy()
    m.apply_translation(-center)

    m.visual = trimesh.visual.ColorVisuals(m, vertex_colors=[200, 200, 200,
    255])

    scene = pyrender.Scene(bg_color=[1.0, 1.0, 1.0, 1.0], ambient_light=[0.6,
    0.6, 0.6])
    scene.add(pyrender.Mesh.from_trimesh(m, smooth=False))

    if overlay_colliders and contract is not None:
        for c in contract.get("colliders", []):
            tm = _make_col_trimesh(c)
            if tm is None:
                continue

            pos = np.array(c.get("localPosition", [0, 0, 0]), dtype=float)
            rot = np.array(c.get("localRotation", [0, 0, 0]), dtype=float)
```

```
            T = np.eye(4)
            T[:3, :3] = _euler_deg_to_rotmat(rot)
            T[:3, 3] = pos
            tm.apply_transform(T)

            tm.visual = trimesh.visual.ColorVisuals(tm, vertex_colors=[255, 60,
    60, 120])
            scene.add(pyrender.Mesh.from_trimesh(tm, smooth=False))

    ext = m.extents
    dist = float(np.linalg.norm(ext) * 1.6 + 0.5)

    cams = {
        "front":  (0, 0, dist),
        "back":   (0, 0, -dist),
        "right":  (dist, 0, 0),
        "left":   (-dist, 0, 0),
        "top":    (0, dist, 1e-3),
        "iso":    (dist * 0.8, dist * 0.6, dist * 0.8),
    }

    camera = pyrender.PerspectiveCamera(yfov=np.pi / 3.0)
    light = pyrender.DirectionalLight(color=np.ones(3), intensity=2.0)

    renderer = pyrender.OffscreenRenderer(out_size[0], out_size[1])

    images: Dict[str, Image.Image] = {}
    for name, eye in cams.items():
        pose = _camera_pose(eye, target=(0, 0, 0), up=(0, 1, 0))
        cam_node = scene.add(camera, pose=pose)
        light_node = scene.add(light, pose=pose)

        color, _ = renderer.render(scene)

        scene.remove_node(cam_node)
        scene.remove_node(light_node)

        images[name] = Image.fromarray(color)

    renderer.delete()
    return images
```

A.3.7 Reporting utilities (`report.py`)

Listing 7: Generation of visual reports for all iterations.

```
from __future__ import annotations
from pathlib import Path
import json
from typing import Dict, Any, List

from PIL import Image
import trimesh
```

```
from render_pyrender import render_multiview

VIEWS = ["front", "back", "left", "right", "top", "iso"]


def save_multiview_images(
    mesh: trimesh.Trimesh,
    contract: Dict[str, Any],
    out_dir: Path,
    out_size=(768, 768),
) -> Dict[str, Path]:
    out_dir.mkdir(parents=True, exist_ok=True)

    imgs = render_multiview(
        mesh,
        out_size=out_size,
        contract=contract,
        overlay_colliders=True,
    )

    saved: Dict[str, Path] = {}
    for v in VIEWS:
        if v not in imgs:
            continue
        p = out_dir / f"{v}.png"
        imgs[v].save(p)
        saved[v] = p

    return saved


def write_final_report(run_dir: Path, result: Dict[str, Any]) -> Path:
    history: List[Dict[str, Any]] = result.get("history", [])
    final = result.get("final") or result.get("best_guess")

    def iter_block(step: Dict[str, Any]) -> str:
        it = int(step.get("iteration", 0))
        contract = step.get("contract", {})
        decision = step.get("decision", {})

        img_base = f"iters/iter_{it:02d}"
        checks = decision.get("checks", {})
        fix_next = decision.get("fix_next", [])

        return f"""
        <div style="border:1px solid #ddd; border-radius:10px; padding:12px;
   margin:14px 0;">
          <h2 style="margin:0 0 8px 0;">Iteration {it}</h2>

          <div><b>Observation (judge):</b></div>
          <div style="margin:6px 0;">
            <b>Decision:</b> {decision.get("decision")} |
            <b>Confidence:</b> {decision.get("confidence")}
          </div>
```

```
        <div style="margin:6px 0;"><b>Reason:</b>
{decision.get("reason","")}</div>

        <details>
          <summary style="cursor:pointer;">Show checks</summary>
          <pre style="background:#f6f6f6;padding:10px;border:1px solid
#eee;">{json.dumps(checks, indent=2)}</pre>
        </details>

        <details>
          <summary style="cursor:pointer;">Show fix_next</summary>
          <pre style="background:#f6f6f6;padding:10px;border:1px solid
#eee;">{json.dumps(fix_next, indent=2)}</pre>
        </details>

        <details>
          <summary style="cursor:pointer;">Show contract JSON</summary>
          <pre style="background:#0b1020; color:#e6e6e6; padding:10px;
border-radius:6px; overflow:auto;">{json.dumps(contract, indent=2)}</pre>
        </details>

        <div style="display:flex; flex-wrap:wrap; gap:10px; margin-top:10px;">
          {''.join([f"<div><div
style='font-size:12px;color:#555;margin-bottom:4px;'>{v}</div><img
src='{img_base}/{v}.png' style='width:240px; border:1px solid #ccc;'></div>"
for v in VIEWS])}
        </div>
      </div>
      """

    html = f"""
    <html>
    <head><meta charset="utf-8"><title>Agent</title></head>
    <body style="font-family:Arial; padding:20px; max-width:1200px;">
      <h1>Agent</h1>

      <div style="margin-bottom:12px;">
        <b>Total iterations:</b> {len(history)}<br>
        <b>Final iteration:</b> {final.get("iteration") if final else "None"}
      </div>

      {''.join(iter_block(s) for s in history)}
    </body>
    </html>
    """.strip()

    report_path = run_dir / "report.html"
    report_path.write_text(html, encoding="utf-8")
    return report_path
```

A.3.8 Contract schema and validation (`schema.py`)

Listing 8: Schema used to validate generated physical contracts.

```
from __future__ import annotations
from dataclasses import dataclass, asdict
from typing import List, Literal, Optional, Dict, Any
import json

"""
Schema + minimal sanity checks.
This is NOT a set of design constraints.
Goal: reject garbage / non-Unity-parsable outputs from the model.
"""

ColliderType = Literal["box", "sphere", "capsule"]


@dataclass
class ColliderSpec:
    type: ColliderType
    localPosition: List[float]
    localRotation: List[float]
    size: List[float]
    isTrigger: bool = False


@dataclass
class PhysicsSpec:
    mass: float = 1.0
    useGravity: bool = True
    isKinematic: bool = False
    linearDamping: float = 0.05
    angularDamping: float = 0.05
    staticFriction: float = 0.6
    dynamicFriction: float = 0.6
    restitution: float = 0.05

    density: Optional[float] = None
    materialGuess: Optional[str] = None


@dataclass
class PhysicalContract:
    colliders: List[ColliderSpec]
    physics: PhysicsSpec

    def to_json(self, indent: int = 2) -> str:
        return json.dumps(asdict(self), indent=indent)

    @staticmethod
    def from_dict(d: Dict[str, Any]) -> "PhysicalContract":
        validate_contract_dict(d)
        cols = [ColliderSpec(**_normalize_collider_dict(c)) for c in
   d["colliders"]]
        phys = PhysicsSpec(**_normalize_physics_dict(d["physics"]))
        return PhysicalContract(colliders=cols, physics=phys)
```

```
def _is_nan(x: float) -> bool:
    return x != x


def _to_float(x: Any) -> Optional[float]:
    if isinstance(x, (int, float)):
        fx = float(x)
        return None if _is_nan(fx) else fx
    if isinstance(x, str):
        try:
            fx = float(x.strip())
            return None if _is_nan(fx) else fx
        except Exception:
            return None
    return None


def _to_bool(x: Any) -> Optional[bool]:
    if isinstance(x, bool):
        return x
    if isinstance(x, str):
        s = x.strip().lower()
        if s in ("true", "1", "yes", "y"):
            return True
        if s in ("false", "0", "no", "n"):
            return False
    return None


def _normalize_vec3(v: Any, name: str) -> List[float]:
    if not isinstance(v, list) or len(v) != 3:
        raise ValueError(f"{name} must be a list of 3 numbers.")
    out: List[float] = []
    for i, x in enumerate(v):
        fx = _to_float(x)
        if fx is None:
            raise ValueError(f"{name}[{i}] must be a number.")
        out.append(float(fx))
    return out


def _normalize_collider_dict(c: Dict[str, Any]) -> Dict[str, Any]:
    t = c.get("type", None)
    if isinstance(t, str):
        t = t.strip().lower()
    if t not in ("box", "sphere", "capsule"):
        raise ValueError("Collider.type must be one of: box, sphere, capsule.")

    lp = _normalize_vec3(c.get("localPosition", None), "Collider.localPosition")
    lr = _normalize_vec3(c.get("localRotation", None), "Collider.localRotation")
    sz = _normalize_vec3(c.get("size", None), "Collider.size")

    if t == "box":
        if sz[0] <= 0 or sz[1] <= 0 or sz[2] <= 0:
            raise ValueError("Box size must be > 0 on all axes.")
```

```
    elif t == "sphere":
        if sz[0] <= 0:
            raise ValueError("Sphere radius (size[0]) must be > 0.")
        sz[1], sz[2] = 0.0, 0.0
    elif t == "capsule":
        if sz[0] <= 0 or sz[1] <= 0:
            raise ValueError("Capsule radius/height must be > 0.")
        sz[2] = 0.0

    it_raw = c.get("isTrigger", False)
    it = _to_bool(it_raw)
    if it is None:
        raise ValueError("Collider.isTrigger must be boolean.")

    c2 = dict(c)
    c2["type"] = t
    c2["localPosition"] = lp
    c2["localRotation"] = lr
    c2["size"] = sz
    c2["isTrigger"] = it
    return c2


def _normalize_physics_dict(p: Dict[str, Any]) -> Dict[str, Any]:
    if not isinstance(p, dict):
        raise ValueError("'physics' must be an object/dict.")

    p2 = dict(p)

    mass = _to_float(p2.get("mass", None))
    if mass is None or mass <= 0:
        raise ValueError("'physics.mass' must be a positive number.")
    p2["mass"] = float(mass)

    for k, default in (("useGravity", True), ("isKinematic", False)):
        b = _to_bool(p2.get(k, default))
        if b is None:
            raise ValueError(f"'physics.{k}' must be boolean.")
        p2[k] = b

    float_defaults = {
        "linearDamping": 0.05,
        "angularDamping": 0.05,
        "staticFriction": 0.6,
        "dynamicFriction": 0.6,
        "restitution": 0.05,
    }
    for k, default in float_defaults.items():
        v = _to_float(p2.get(k, default))
        if v is None:
            raise ValueError(f"'physics.{k}' must be a number.")
        p2[k] = float(v)

    if "density" in p2 and p2["density"] is not None:
        d = _to_float(p2["density"])
```

```
        if d is None or d <= 0:
            raise ValueError("'physics.density' must be a positive number when
    provided.")
        p2["density"] = float(d)

    if "materialGuess" in p2 and p2["materialGuess"] is not None:
        if not isinstance(p2["materialGuess"], str):
            raise ValueError("'physics.materialGuess' must be a string when
    provided.")

    return p2


def validate_contract_dict(d: Dict[str, Any]) -> None:
    if not isinstance(d, dict):
        raise ValueError("Contract must be a dict.")
    if "colliders" not in d or "physics" not in d:
        raise ValueError("Contract must contain 'colliders' and 'physics'.")

    cols = d["colliders"]
    phys = d["physics"]

    if not isinstance(cols, list) or len(cols) == 0:
        raise ValueError("'colliders' must be a non-empty list.")

    for c in cols:
        if not isinstance(c, dict):
            raise ValueError("Each collider must be an object/dict.")
        _normalize_collider_dict(c)

    _normalize_physics_dict(phys)
```